

课程学期项目主题目录

2026年春季学期项目主题中英对照翻译文档

领域 1：城市出租车与网约车系统 / Field 1: Urban Taxi & Ride-Sharing Systems

1. 时空自主出租车调度系统 (MOD) / Spatio-Temporal Autonomous Taxi Dispatching (MOD)

● 目标 (The Goal):

优化车队效用，利用空间索引最小化乘客等待时间。

Original: Optimize fleet utility and minimize passenger wait times using spatial indexing.

● 详细任务 (Detailed Tasks):

- 索引设计：在 SQLite 中实现 R-Tree 索引，以存储通过 TraCI 更新的移动出租车坐标 (x, y)。

Original: Index Design: Implement an R-Tree index in SQLite to store moving taxi coordinates (x, y) updated via TraCI.

- 请求处理：生成随机的乘客出行请求；使用 K-最近邻 (KNN) 查询来查找最近的空闲出租车。

Original: Request Handling: Generate stochastic passenger trip requests; use a K-Nearest Neighbor (KNN) query to find the closest idle taxis.

- 调度逻辑：编写一个 Python 控制器，在数据库中分配出租车，并使用 TraCI 的 setRoute 导航到乘客处。

Original: Dispatch Logic: Write a Python controller that assigns the taxi in the DB and uses TraCI's setRoute to navigate to the passenger.

- 可用性管理：使用原子事务将出租车状态从 IDLE（空闲）更新为 PICKUP（接客）再到 OCCUPIED（载客）。

Original: Availability Management: Use atomic transactions to update taxi status from IDLE to PICKUP to OCCUPIED.

- 再平衡：定期查询数据库中请求密度高但出租车供应低的区域，并将空置出租车“再平衡”到这些区域。

Original: Rebalancing: Periodically query the DB for areas with high request density but low taxi supply and "rebalance" empty taxis to those zones.

● 性能评估规模与主题指标 (Evaluation Scale & Metrics):

- 路网：3 km × 3 km 城市网格 (>500 条边)。

Original: Network: 3 km × 3 km urban grid (>500 edges).

- 规模：2,000 辆背景车辆；300 辆活跃出租车；1,500 个乘客请求。

Original: Scale: 2,000 background vehicles; 300 active taxis; 1,500 passenger requests.

- 时间：3,600秒 (1小时) 高峰需求。

Original: Time: 3,600s (1 hour) peak demand.

- 主题指标：平均乘客等待时间 (AWT)

和出租车空驶率。分析随着车队密度增加三倍，空间索引深度如何与调度成功率相关联。

Original: Topic Metrics: Average Passenger Wait Time (AWT) and Taxi Empty-Mile Ratio. Analyze how the spatial index depth correlates with dispatch success rates as fleet density triples.

2. 拼车匹配中的事务完整性 / Transactional Integrity in Shared-Ride Matching

● 目标 (The Goal):

在不超订的前提下，通过复杂的关联约束最大化车辆利用率。

Original: Maximize vehicle occupancy through complex relational constraints without overbooking.

● 详细任务 (Detailed Tasks):

- 关系模式：设计 Vehicles（车辆）、Current_Passengers（当前乘客）和 Trip_Requests（出行请求）的关系模式，并包含座位数约束。

Original: Relational Schema: Design a schema for Vehicles, Current_Passengers, and Trip_Requests with seat-count constraints.

- 重叠匹配：编写一个 SQL 查询，在特定时间窗口 Δt 内识别具有重叠起点-终点路径的乘客。

Original: Overlap Matching: Write a SQL query that identifies passengers with overlapping origin-destination paths within a specific time window Δt .

- 容量强制执行：使用 SQL CHECK 约束和事务，确保在 当前载客量 + 新请求人数 > 车辆容量 时，车辆绝不接受新乘客。

Original: Capacity Enforcement: Use SQL CHECK constraints and transactions to ensure a vehicle never accepts a passenger if $\text{current_load} + \text{new_request} > \text{capacity}$.

- 动态重路由：只要数据库中确认了新的拼车匹配，就使用 TraCI 向车辆路径中添加中间停靠点。

Original: Dynamic Rerouting: Use TraCI to add intermediate stops to a vehicle's route whenever a new shared-ride match is confirmed in the DB.

- 偏好过滤：在 SQL 匹配查询中加入用户偏好（例如“最多 1 个额外停靠点”）。

Original: Preference Filtering: Include user preferences (e.g., "maximum 1 extra stop") in the SQL matching query.

● 性能评估规模与主题指标 (Evaluation Scale & Metrics):

- 路网：5km × 5km 混合用途区域。

Original: Network: 5km × 5km mixed-use area.

- 规模：500 辆拼车车辆；3,000 个出行请求；每辆车 4 个座位。

Original: Scale: 500 shared-ride vehicles; 3,000 trip requests; 4 seats per vehicle.

- 时间：7,200秒（2小时窗口）。

Original: Time: 7,200s (2-hour window).

- 主题指标：平均车辆载客率 (AVO) 和总车辆行驶里程 (VMT) 减少量。评估在高峰匹配爆发期间由事务超时引起的“拒绝率”。

Original: Topic Metrics: Average Vehicle Occupancy (AVO) and Total Vehicle Miles Traveled (VMT) Reduction. Evaluate the "Rejection Rate" caused by transaction timeouts during peak matching bursts.

3. 动态激增定价与供需平衡 / Dynamic Surge Pricing and Demand-Supply Equilibrium

- **目标 (The Goal):**

利用分析型窗口函数，通过经济激励措施平衡城市交通需求。

Original: Use analytical window functions to balance urban transport demand via economic incentives.

- **详细任务 (Detailed Tasks):**

- 区域聚合：将 SUMO 地图划分为不同的区域；使用 SQL 窗口函数计算过去 T 分钟内的需求密度。

Original: Zonal Aggregation: Divide the SUMO map into districts; use SQL window functions to calculate demand density in the last T minutes.

- 定价逻辑：实现一个基于 Python 的定价引擎，根据每个区域的供需比更新 Price_Index（价格指数）表。

Original: Price Logic: Implement a Python-based pricing engine that updates a Price_Index table based on the demand/supply ratio of each district.

- 智能体行为：为 SUMO 车辆分配“支付意愿”属性；如果价格过高，车辆会查询数据库并选择“取消”或“绕行”。

Original: Agent Behavior: Assign a "Willingness-to-Pay" attribute to SUMO vehicles; vehicles query the DB and choose to "cancel" or "divert" if prices are too high.

- 收入追踪：记录所有已完成的交易以评估平台盈利能力。

Original: Revenue Tracking: Log all completed transactions to evaluate platform profitability.

- 弹性分析：使用数据库存储历史价格与需求数据，以校准定价模型。

Original: Elasticity Analysis: Use the DB to store historical price-vs-demand data to calibrate the pricing model.

- **性能评估规模与主题指标 (Evaluation Scale & Metrics):**

- 路网：市中心的 10 个不同区域。

Original: Network: 10 distinct districts in a city center.

- 规模：5,000 个具有不同价格敏感度的智能体。

Original: Scale: 5,000 agents with varying price sensitivities.

- 时间：14,400秒（4小时早高峰）。

Original: Time: 14,400s (4-hour morning rush).

- 主题指标：拥挤平滑因子（各区域平均速度 of 方差）和请求履行率。通过将数据库记录的价格与智能体“分流”率相关联来衡量价格弹性。

Original: Topic Metrics: Congestion Smoothing Factor (variance in average speed across districts) and Request Fulfillment Rate. Measure price elasticity by correlating DB-logged prices with agent "Divergence" rates.

领域 2：车辆导航与基础设施 / Field 2: Vehicle Navigation & Infrastructure

4. 基于持久边权重的拥堵感知导航 / Congestion-Aware Navigation with Persistent Edge-Weights

• 目标 (The Goal):

通过使用数据库作为动态图权重仓库，改善全城范围的旅行时间。

Original: Improve city-wide travel times by using the DB as a dynamic graph-weight repository.

• 详细任务 (Detailed Tasks):

- 图模式：在 SQLite 中表示道路网格 (Edges/边、Nodes/节点、Weights/权重)。

Original: Graph Schema: Represent the road network in SQLite (Edges, Nodes, Weights).

- 实时更新：使用 TraCI 测量实时速度；每隔 N 秒更新一次 Weights 表。

Original: Live Updates: Use TraCI to measure real-time speeds; update the Weights table every N seconds.

- 外部路径规划：使用 Python 运行 Dijkstra/A* 算法，从 SQLite 中获取权重，而不是使用 SUMO 内部的静态路由。

Original: External Pathfinding: Use Python to run a Dijkstra/A* algorithm that fetches weights from SQLite rather than using SUMO's internal static routing.

- 路径注入：通过 TraCI 强制“智能车辆”行驶在这些由数据库计算出的路径上。

Original: Route Injection: Force "Smart Vehicles" onto these DB-calculated paths via TraCI.

- 事件管理：手动在数据库中关闭某条边（模拟事故），并观察查询驱动的导航如何重新路由交通。

Original: Incident Management: Manually close an edge in the DB (simulating an accident) and observe how the query-driven navigation reroutes traffic.

• 性能评估规模与主题指标 (Evaluation Scale & Metrics):

- 路网：城市级路网 (>10,000 条边)。

Original: Network: City-scale network (>10,000 edges).

- 规模：5,000 辆从数据库获取权重的“智能车辆”；2,000 辆“静态”车辆。

Original: Scale: 5,000 "Smart Vehicles" fetching weights from DB; 2,000 "Static" vehicles.

- 时间：3,600秒。

Original: Time: 3,600s.

- 主题指标：平均旅程时间 (AJT)

和全局路网吞吐量。比较在模拟事故阻断主要干道后的“路线收敛”时间（系统适应的速度）。

Original: Topic Metrics: Average Journey Time (AJT) and Global Network Throughput. Compare the "Route Convergence" time (how fast the system adapts) after a simulated accident blocks a major arterial.

5. 城市交通 “数字孪生” 与事故重现 / Urban Traffic "Digital Twin" & Forensic Replay

- **目标 (The Goal):**

实现事故取证和交通审计的高保真状态重建。

Original: Achieve high-fidelity state reconstruction for accident forensics and traffic auditing.

- **详细任务 (Detailed Tasks):**

- 遥测记录：设计一个高吞吐量的模式，记录每辆车在每一步的 (ID, x, y, speed, angle)。

Original: Telemetry Logging: Design a high-throughput schema to log (ID, x, y, speed, angle) for every vehicle at every step.

- 批量摄取：使用 `executemany` 优化 Python 到 SQLite 的管道，实现实时记录。

Original: Batch Ingestion: Optimize the Python-to-SQLite pipeline using `executemany` for real-time logging.

- 冲突检测：使用 SQL 触发器识别 “险些酿成事故” （当车速 > 0 时，车辆间距 $d < 2m$ ）。

Original: Conflict Detection: Use SQL triggers to identify "Near-Misses" (vehicles within $d < 2m$ while speed > 0).

- 状态重放：编写一个脚本，从数据库读取特定时间戳，并在新的 SUMO 实例中精确重建该车辆分布。

Original: State Replay: Write a script that reads a specific timestamp from the DB and recreates that exact vehicle distribution in a new SUMO instance.

- 趋势分析：聚合遥测数据，识别 “急刹车” 频率最高的道路。

Original: Trend Analysis: Aggregate telemetry to identify roads with the highest "sudden braking" frequency.

- **性能评估规模与主题指标 (Evaluation Scale & Metrics):**

- 路网：2km × 2km 高密度市中心。

Original: Network: 2km × 2km high-density downtown.

- 规模：1,500 辆车。

Original: Scale: 1,500 vehicles.

- 时间：1,800秒（30分钟高分辨率时间窗口）。

Original: Time: 1,800s (30-minute high-resolution window).

- 主题指标：取证重建准确度（x, y 坐标的均方误差）和事件检测召回率。衡量存储占用空间与在 50cm 精度内重建碰撞能力之间的平衡。

Original: Topic Metrics: Forensic Reconstruction Accuracy (Mean Squared Error of x,y coordinates) and Incident Detection Recall. Measure the storage footprint vs. the ability to reconstruct a collision within 50cm of accuracy.

6. 基于关系逻辑的自适应交通信号控制 (ATSC) / Adaptive Traffic Signal Control (ATSC) via Relational Logic

• 目标 (The Goal):

通过将红绿灯视为数据库驱动的执行器，减少交叉口延误。

Original: Reduce intersection delay by treating traffic lights as DB-driven actuators.

• 详细任务 (Detailed Tasks):

- 基础设施映射：在数据库中存储所有交通信号灯相位 and 感应线圈的关联。

Original: Infrastructure Mapping: Store all traffic light phases and induction loop associations in the DB.

- 传感器流：将感应线圈的计数流式传输到 SQLite 的 Sensor_Data 表中。

Original: Sensor Streaming: Stream induction loop counts into an SQLite Sensor_Data table.

- 压力逻辑：通过 SQL 查询实现“最大压力”控制器，确定等待车辆数最多的相位。

Original: Pressure Logic: Implement a "Max-Pressure" controller via SQL queries that determine the phase with the highest waiting volume.

- 信号执行：根据数据库的决策，通过 TraCI 更新 SUMO 的交通信号灯状态。

Original: Signal Actuation: Update SUMO traffic light states via TraCI based on the DB's decision.

- 自适应反馈：使用数据库中的历史吞吐量数据来调整“最小绿灯时间”参数。

Original: Adaptive Feedback: Use historical throughput data in the DB to adjust the "Minimum Green Time" parameters.

• 性能评估规模与主题指标 (Evaluation Scale & Metrics):

- 路网：10 个互连的有信号控制的交叉口。

Original: Network: 10 interconnected signalized intersections.

- 规模：2,500 辆流经该走廊的车辆。

Original: Scale: 2,500 vehicles flowing through a corridor.

- 时间：3,600秒高峰时段。

Original: Time: 3,600s peak hour.

- 主题指标：车均延误和最大排队长度。评估与固定时间 Webster 周期基准相比的“系统吞吐量”。

Original: Topic Metrics: Average Delay per Vehicle and Maximum Queue Length. Evaluate the "Systemic Throughput" compared to a fixed-time Webster cycle baseline.

7. 预测性道路维护与资产管理 / Predictive Road Maintenance and Asset Management

- **目标 (The Goal):**

通过模拟累积损伤，平衡道路寿命与交通流量。

Original: Balance road longevity with traffic flow by simulating cumulative damage.

- **详细任务 (Detailed Tasks):**

- 耐用度模式：创建一个 Edge_Health (边健康度) 表；道路根据车辆重量和流量降低 “健康度” 。

Original: Durability Schema: Create an Edge_Health table; roads lose "health" based on vehicle weight and volume.

- 累积损伤：每当一辆 “卡车” 类车辆在 TraCI 中通过某条边时，执行 SQL UPDATE 来递减健康度。

Original: Cumulative Damage: Every time a "Truck" class vehicle passes an edge in TraCI, execute a SQL UPDATE to decrement health.

- 故障模拟：当健康度降为零时，使用 TraCI 关闭该车道或将限速降至 10km/h。

Original: Failure Simulation: When health drops to zero, use TraCI to close the lane or reduce the speed limit to 10km/h.

- 预测性查询：使用 SQL 识别 “在未来 1000 步内最可能失效的前 5 条边” 。

Original: Predictive Querying: Use SQL to identify the "Top 5 edges likely to fail in the next 1000 steps."

- 维护计划：设计一个基于 SQL 的调度程序，优先修复高流量道路。

Original: Maintenance Planning: Design a SQL-based scheduler that prioritizes repairs on high-traffic roads.

- **性能评估规模与主题指标 (Evaluation Scale & Metrics):**

- 路网：拥有 20 个桥梁/隧道资产的 50km 高速公路网。

Original: Network: 50km highway network with 20 bridge/tunnel assets.

- 规模：总计 10,000 辆车次通过（在加速的多日周期内模拟）。

Original: Scale: 10,000 total vehicle passes (simulated over accelerated multi-day cycles).

- 时间：43,200秒（12小时加速窗口）。

Original: Time: 43,200s (12-hour accelerated window).

- 主题指标：路网可用时间 (%) 和基础设施健康度方差。分析数据库驱动的维护优先级如何影响路段的 “生命周期成本” 。

Original: Topic Metrics: Network Uptime (%) and Infrastructure Health Variance. Analyze how the DB-driven maintenance priority affects the "Life-Cycle Cost" of the road segments.

领域 3：物流与新兴出行方式 / Field 3: Logistics & Emerging Mobility

8. 电动汽车车队充电物流与电网平衡 / EV Fleet Charging Logistics and Grid Balancing

- **目标 (The Goal):**

管理电动汽车车队以确保旅程完成，同时避免充电站拥堵。

Original: Manage electric vehicle fleets to ensure trip completion while avoiding charging station congestion.

- **详细任务 (Detailed Tasks):**

- 电池追踪：在关系表中追踪所有电动汽车的荷电状态 (SoC)。

Original: Battery Tracking: Track State-of-Charge (SoC) for all EVs in a relational table.

- 低电量事件：使用触发器/轮询器检测 SoC < 15% 的电动汽车。

Original: Low-Battery Events: Use a trigger/poller to detect EVs with SoC < 15%.

- 充电队列：为充电站实现一个基于 SQL 的队列，优先处理 SoC 最低的车辆。

Original: Charging Queue: Implement a SQL-based queue for charging stations, prioritizing vehicles with the lowest SoC.

- 重定向：使用 TraCI 将电动汽车发送到数据库分配的充电站。

Original: Rerouting: Use TraCI to send EVs to the station assigned by the DB.

- 负荷预测：聚合当前的电动汽车目的地，预测未来一小时内哪些充电站会超载。

Original: Load Prediction: Aggregate current EV destinations to predict which charging stations will be overloaded in the next hour.

- **性能评估规模与主题指标 (Evaluation Scale & Metrics):**

- 路网：拥有 25 个充电站的城市区域。

Original: Network: Urban area with 25 charging stations.

- 规模：3,000 辆总车流中的 600 辆电动汽车。

Original: Scale: 600 EVs in a 3,000 vehicle total population.

- 时间：43,200秒（12小时模拟周期）。

Original: Time: 43,200s (12-hour simulation cycle).

- 主题指标：车队停机时间（充电等待时间）和电网峰谷比。评估“里程焦虑”因素——未能到达充电站的电动汽车百分比。

Original: Topic Metrics: Fleet Downtime (Charging Wait Time) and Grid Peak-to-Average Ratio. Evaluate the "Range Anxiety" factor—the percentage of EVs that failed to reach a charger.

9. 多模态 “出行即服务” (MaaS) 集成 / Multi-Modal "Mobility-as-a-Service" (MaaS) Integration

● 目标 (The Goal):

通过同步不同的交通模式，优化点到点的旅行时间。

Original: Optimize door-to-door travel times by synchronizing disparate transit modes.

● 详细任务 (Detailed Tasks):

- MaaS 模式：将公交车时刻表与实时出租车可用性结合在单个数据库中。

Original: MaaS Schema: Combine bus schedules with real-time taxi availability in a single database.

- 多模态路由：编写一个 SQL 连接，为用户寻找最快的 “出租车到公交车” 换乘点。

Original: Intermodal Routing: Write a SQL join that finds the fastest "Taxi-to-Bus" transfer point for a user.

- 动态同步：使用 TraCI 追踪公交车延误；数据库应自动提醒出租车 “等待” 或 “加速” 以赶上公交车。

Original: Dynamic Sync: Use TraCI to track bus delays; the DB should automatically alert the taxi to "wait" or "speed up" to meet the bus.

- 用户个性化：将用户偏好（例如 “相比出租车更喜欢公交车” ）存储为匹配查询中的权重。

Original: User Personalization: Store user preferences (e.g., "Prefer Bus over Taxi") as weights in the matching query.

- 吞吐量报告：使用 SQL 计算平均 “多模态换乘时间” 。

Original: Throughput Reporting: Use SQL to calculate the average "Intermodal Transfer Time."

● 性能评估规模与主题指标 (Evaluation Scale & Metrics):

- 路网：15 条公交线；120 个公交停靠站；250 辆出租车。

Original: Network: 15 bus lines; 120 bus stops; 250 taxis.

- 规模：5,000 个独特的用户旅程；3,000 辆背景车辆。

Original: Scale: 5,000 unique user journeys; 3,000 background cars.

- 时间：7,200秒。

Original: Time: 7,200s.

- 主题指标：平均多模态换乘时间和总行程可靠性 (%)。比较通过数据库同步使公交车 “等待” 晚到的出租车乘客时的等待时间。

Original: Topic Metrics: Average Intermodal Transfer Time and Total Journey Reliability (%). Compare wait times when buses "wait" for late-arriving taxi passengers via DB synchronization.

10. 基于递归层级的连接车队管理 / Connected Platoon Management via Recursive Hierarchies

• 目标 (The Goal):

使用递归数据结构管理高密度车辆串，以增加道路容量。

Original: Use recursive data structures to manage high-density vehicle strings for increased road capacity.

• 详细任务 (Detailed Tasks):

- 层级模式：使用自引用表 (Vehicle_ID, Leader_ID) 表示车队 (Platoons) 。

Original: Hierarchy Schema: Represent platoons using a self-referential table (Vehicle_ID, Leader_ID).

- 加入/离开逻辑：使用 TraCI 检测车辆邻近度；执行 SQL 事务将跟随车 “加入” 到领航车。

Original: Join/Leave Logic: Use TraCI to detect proximity; execute a SQL transaction to "join" a follower to a leader.

- 递归更新：使用递归公用表表达式 (Recursive CTE) 将速度变化从领航车传播到数据库中的所有 N 个跟随车。

Original: Recursive Update: Use a Recursive CTE to propagate a speed change from the leader to all N followers in the DB.

- 解体管理：当领航车驶离高速公路时，使用 SQL 查询将第二辆车提升为领航车并更新层级关系。

Original: Dissolution Management: When a leader exits the highway, use a SQL query to promote the second vehicle to leader and update the hierarchy.

- 效率审计：记录车队车辆与非车队车辆的 “车头时距/跟车距离 (Headway)” 。

Original: Efficiency Audit: Log the "Headway" distance of platooned vs. non-platooned vehicles.

• 性能评估规模与主题指标 (Evaluation Scale & Metrics):

- 路网：10km, 3 车道高速公路。

Original: Network: 10km, 3-lane highway.

- 规模：500 辆重型卡车；1,000 辆乘用车。

Original: Scale: 500 heavy-duty trucks; 1,000 passenger cars.

- 时间：3,600秒。

Original: Time: 3,600s.

- 主题指标：车道吞吐量 (辆/车道/小时)

和车队稳定性 (激波阻尼)。衡量 “队列稳定性” —— 制动事件在数据库管理的层级中向后传播的距离。

Original: Topic Metrics: Lane Throughput (Veh/Lane/Hour) and Platoon Stability (damping of shockwaves). Measure the "String Stability"—how far back a braking event propagates through the DB-managed hierarchy.

领域 4：安全与政策执行 / Field 4: Safety & Policy Enforcement

11. 应急服务车辆 (ESV) 通道抢占 / Emergency Service Vehicle (ESV) Corridor Preemption

● 目标 (The Goal):

通过抢先清理交通，最小化应急车辆的响应时间。

Original: Minimize response times for emergency vehicles by preemptively clearing traffic.

● 详细任务 (Detailed Tasks):

- 优先级模式：维护一张活跃的 ESV 调度及其当前 GPS 坐标的表格。

Original: Priority Schema: Maintain a table of active ESV dispatches and their current GPS coordinates.

- 抢占区域：使用 R-Tree 查询识别 ESV 前方 500m 内的所有交通信号灯和民用车辆。

Original: Preemption Zone: Use an R-Tree query to identify all traffic lights and civilian vehicles 500m ahead of the ESV.

- 主动控制：使用 TraCI 强制民用车辆靠外侧车道行驶，并为 ESV 通道将交通信号灯变绿。

Original: Active Control: Use TraCI to force civilian vehicles to the outer lanes and turn traffic lights green for the ESV corridor.

- 冲突解决：使用数据库管理两辆从不同方向驶向同一交叉口的 ESV。

Original: Conflict Resolution: Use the DB to manage two ESVs approaching the same intersection from different directions.

- 影响分析：记录在抢占期间给民用交通造成的延误。

Original: Impact Analysis: Log the delays caused to civilian traffic during the preemption.

● 性能评估规模与主题指标 (Evaluation Scale & Metrics):

- 路网：4km × 4km 城市中心。

Original: Network: 4km × 4km urban center.

- 规模：20 辆活跃的 ESV；2,500 辆民用车辆。

Original: Scale: 20 active ESVs; 2,500 civilian vehicles.

- 时间：3,600秒（随机事故调度）。

Original: Time: 3,600s (randomized incident dispatches).

- 主题指标：ESV

平均响应时间及民用车辆延误惩罚（秒/车辆）。绘制响应时间减少量与数据库中“拥堵波及效应”距离的关系图。

Original: Topic Metrics: ESV Mean Response Time and Civilian Delay Penalty (seconds/vehicle). Plot the response time reduction vs. the "Congestion Ripple Effect" distance in the DB.

12. 隐私保护的轨迹掩蔽与数据效用 / Privacy-Preserving Trajectory Masking & Data Utility

● 目标 (The Goal):

在确保个体车辆隐私的同时提供有用的交通分析。

Original: Provide useful traffic analytics while ensuring individual vehicle privacy.

● 详细任务 (Detailed Tasks):

- 双重模式：存储 Raw_Data (原始数据, 内部) 和 Anonymized_Data (匿名化数据, 公开视图)。

Original: Dual Schema: Store Raw_Data (internal) and Anonymized_Data (public view).

- 噪声注入：在提交到公开数据库之前，对 TraCI 坐标应用拉普拉斯噪声。

Original: Noise Injection: Apply Laplacian noise to TraCI coordinates before committing to the public DB.

- K-匿名性：实现 SQL 视图，仅在区域内至少有 K 辆车时显示交通计数。

Original: K-Anonymity: Implement SQL views that only show traffic counts if at least K vehicles are in a zone.

- 效用测试：在两个表上运行交通密度查询并比较结果。

Original: Utility Testing: Run a traffic density query on both tables and compare results.

- 隐私审计：尝试根据公开数据库中的移动模式“重新识别”特定的车辆 ID。

Original: Privacy Audit: Attempt to "re-identify" a specific vehicle ID based on movement patterns in the public DB.

● 性能评估规模与主题指标 (Evaluation Scale & Metrics):

- 路网：3 km × 3 km 干道网。

Original: Network: 3 km × 3 km arterial grid.

- 规模：4,000 辆移动车辆。

Original: Scale: 4,000 moving vehicles.

- 时间：3,600秒。

Original: Time: 3,600s.

- 主题指标：查询效用（密度图的均方根误差 RMSE）与隐私泄漏概率。为交通规划者衡量“噪声与准确性”之间的权衡。

Original: Topic Metrics: Query Utility (RMSE of Density Maps) and Privacy Disclosure Probability. Measure the "Noise vs. Accuracy" trade-off for traffic planners.

13. 智能车位分配与巡航减少 / Smart Parking Allocation & Cruising Reduction

● 目标 (The Goal):

通过实时预约数据库，消除“为寻找停车位而漫游/巡航”的现象。

Original: Eliminate "cruising for parking" through a real-time reservation database.

● 详细任务 (Detailed Tasks):

- 停车模式：将 SUMO 停车区域映射到 Parking_Spots 表。

Original: Parking Schema: Map SUMO parking areas to a Parking_Spots table.

- 巡航检测：识别在目的地周围绕圈行驶的车辆，并记录其状态。

Original: Cruising Detection: Identify vehicles circling a destination via TraCI and log their status.

- 预约引擎：当车辆进入城市时，使用 SQL 分配并预留最近的车位。

Original: Reservation Engine: When a vehicle enters the city, use SQL to assign and reserve the nearest spot.

- 价格响应：在数据库中实现动态定价；如果占用率 > 90%，价格上涨，车辆可能会选择停在更远的地方。

Original: Price Response: Implement dynamic pricing in the DB; if occupancy is > 90%, prices increase, and vehicles may choose to park further away.

- 库存审计：记录没有预约的车辆所浪费的总寻找时间和燃料。

Original: Inventory Audit: Log total search time and fuel wasted by vehicles without a reservation.

● 性能评估规模与主题指标 (Evaluation Scale & Metrics):

- 路网：800 个路侧停车位；50 个路外停车场。

Original: Network: 800 on-street parking spots; 50 off-street lots.

- 规模：2,500 辆在市中心有目的地的车辆。

Original: Scale: 2,500 vehicles with destinations in the CBD.

- 时间：7,200秒。

Original: Time: 7,200s.

- 主题指标：寻找车位的平均时间及巡航相关的 VMT。评估通过消除循环巡航实现的“系统级燃料节省”。

Original: Topic Metrics: Average Search Time for Parking and Cruising-related VMT. Evaluate the "System-wide Fuel Savings" achieved by eliminating circular cruising.

14. 基于险些碰撞自连接的道路安全审计 / Road Safety Auditing via Near-Miss Self-Joins

● 目标 (The Goal):

在事故发生前，使用数据库主动识别危险的道路设计。

Original: Use the DB to proactively identify dangerous road designs before accidents occur.

● 详细任务 (Detailed Tasks):

- 邻近度记录：记录每辆车在每一步的位置。

Original: Proximity Logging: Log every vehicle's position at every step.

- 自连接查询：运行一个查询，找出所有间距 $d < 1\text{m}$ 且速度 > 0 的车辆对 (V_1, V_2)。

Original: Self-Join Query: Run a query that finds all pairs of vehicles (V_1, V_2) where distance $d < 1\text{m}$ and speed > 0 .

- 热点地图：使用 SQL 计算每个交叉口的险些碰撞次数。

Original: Hotspot Mapping: Use SQL to count near-misses per intersection.

- 时间相关性：将险些碰撞的频率与数据库中存储的特定交通信号相位相关联。

Original: Temporal Correlation: Correlate near-miss frequency with specific traffic signal phases stored in the DB.

- 缓解测试：修改 SUMO 路网（例如添加停止标志）并重新运行审计。

Original: Mitigation Test: Modify the SUMO network (e.g., add a stop sign) and re-run the audit.

● 性能评估规模与主题指标 (Evaluation Scale & Metrics):

- 路网：5 个具有不同几何形状的主要交叉口。

Original: Network: 5 major intersections with varying geometries.

- 规模：3,000 辆车。

Original: Scale: 3,000 vehicles.

- 时间：3,600秒高密度模拟。

Original: Time: 3,600s high-density simulation.

- 主题指标：冲突频率指数 (CFI) 和碰撞时间 (TTC) 分布。识别数据库中的特定信号相位是否与险些碰撞次数暴增 50% 相关联。

Original: Topic Metrics: Conflict Frequency Index (CFI) and Time-to-Collision (TTC) Distribution. Identify if a specific signal phase in the DB correlates with a 50% spike in near-misses.

15. 基于距离的动态道路收费 / Distance-Based Dynamic Road Tolling

- **目标 (The Goal):**

利用透明的、由数据库驱动的财务交易，减少市中心的拥堵。

Original: Reduce congestion in city centers using transparent, DB-driven financial transactions.

- **详细任务 (Detailed Tasks):**

- 钱包模式：管理用户余额和车辆进出记录。

Original: Wallet Schema: Manage user balances and vehicle entry/exit logs.

- 动态定价：根据实时拥堵水平（区域内的车辆计数）计算通行费。

Original: Dynamic Pricing: Calculate tolls based on real-time congestion levels (count of vehicles in the zone).

- 交易处理：使用 ACID 事务扣除资金并记录移动。

Original: Transaction Processing: Use ACID transactions to deduct funds and log the movement.

- 分流分析：追踪车辆是否采取更长、不收费的路线来躲避费用。

Original: Diversion Analysis: Track if vehicles are taking longer, non-tolled routes to avoid fees.

- 收入视图：生成每小时城市收入与车辆平均速度的对比报告。

Original: Revenue View: Generate an hourly report of city revenue vs. average vehicle speed.

- **性能评估规模与主题指标 (Evaluation Scale & Metrics):**

- 路网：2km × 2km 的收费市中心 (CBD) 区域。

Original: Network: 2km × 2km tolled CBD zone.

- 规模：4,000 辆进出车辆。

Original: Scale: 4,000 vehicles entering/exiting.

- 时间：10,800秒（3小时上午高峰）。

Original: Time: 10,800s (3-hour morning peak).

- 主题指标：高峰车速提升及区域进入弹性。评估“收入延误比”——每收一美元通行费减少了多少延误。

Original: Topic Metrics: Peak Speed Increase and Zone Entry Elasticity. Evaluate the "Revenue-to-Delay" ratio—how much delay is reduced per dollar of toll collected.

领域 5：物流与网络韧性 / Field 5: Logistics & Network Resilience

16. 最后一公里物流：卡车-无人机交会协调 / Last-Mile Logistics: Truck-Drone Rendezvous Coordination

- **目标 (The Goal):**

通过在共享的空间数据库中协调卡车和无人机，最大化配送效率。

Original: Maximize delivery efficiency by coordinating trucks and drones in a shared spatial DB.

- **详细任务 (Detailed Tasks):**

- 智能体模式：追踪卡车和无人机的位置及电池电量。

Original: Agent Schema: Track locations and battery levels for Trucks and Drones.

- 交会逻辑：使用 SQL 寻找卡车与无人机相遇进行电池更换/重新装载的最佳 SUMO 边。

Original: Rendezvous Logic: Use SQL to find the best SUMO edge for a truck to meet a drone for battery swaps/reloads.

- 移交逻辑：使用事务将“包裹”的所有权从卡车转移到无人机。

Original: Handover Logic: Use transactions to transfer "Parcel" ownership from Truck to Drone.

- 路径同步：如果无人机发生延误（从数据库查询），使用 TraCI 让卡车在交会点等待。

Original: Path Sync: Use TraCI to hold the truck if the drone is delayed (queried from DB).

- 效率审计：将“每小时配送的包裹数”与仅卡车配送的基准进行对比。

Original: Efficiency Audit: Compare "Parcels Delivered per Hour" against a truck-only baseline.

- **性能评估规模与主题指标 (Evaluation Scale & Metrics):**

- 路网：2km × 2km 住宅区。

Original: Network: 2km × 2km residential area.

- 规模：30 辆卡车；150 架无人机；1,000 个包裹。

Original: Scale: 30 trucks; 150 drones; 1,000 parcels.

- 时间：14,400秒（4小时窗口）。

Original: Time: 14,400s (4-hour window).

-

主题指标：配送吞吐量（包裹/小时）和每次配送的能源成本。通过数据库状态日志衡量卡车等待无人机的“空闲时间”。

Original: Topic Metrics: Delivery Throughput (Parcels/Hour) and Energy Cost per Delivery. Measure the "Idle Time" for trucks waiting on drones via DB-status logs.

17. 基于预测需求的微出行再平衡 / Micromobility Rebalancing via Predictive Demand

● 目标 (The Goal):

利用 SQL 中的历史需求模式，维持共享单车/滑板车的可用性。

Original: Maintain e-scooter availability using historical demand patterns in SQL.

● 详细任务 (Detailed Tasks):

- 滑板车模式：追踪 ID、位置和状态（可用/使用中）。

Original: Scooter Schema: Track ID, position, and status (available/in-use).

- 需求映射：记录所有“用户请求”和成功的租赁。

Original: Demand Mapping: Log all "User Requests" and successful rentals.

- 失衡检测：使用 SQL 查找需求 > 供应的区域。

Original: Imbalance Detection: Use SQL to find zones where demand > supply.

- 再平衡任务：在 SUMO 中派遣一辆“再平衡卡车”，将滑板车从低需求区域移至高需求区域。

Original: Rebalancing Task: Dispatch a "Rebalancing Truck" in SUMO to move units from low-demand to high-demand areas.

- 可用性审计：计算“服务水平”（在 100m 内找到滑板车的用户比例）。

Original: Availability Audit: Calculate the "Service Level" (% of users finding a scooter within 100m).

● 性能评估规模与主题指标 (Evaluation Scale & Metrics):

- 路网：3 km × 3 km 大学/商业区。

Original: Network: 3 km × 3 km university/business district.

- 规模：500 辆电子滑板车；80 个停靠站；2,000 个用户。

Original: Scale: 500 e-scooters; 80 docking stations; 2,000 users.

- 时间：86,400秒（完整 24 小时周期）。

Original: Time: 86,400s (Full 24-hour cycle).

- 主题指标：服务可用率 (%) 和再平衡卡车里程数。评估应用预测性 SQL 模型后“服务水平”的改善情况。

Original: Topic Metrics: Service Availability Rate (%) and Rebalancing Truck Odometer. Evaluate the "Service Level" improvement after applying the predictive SQL model.

18. 动态公交专用道 (DBL) 激活策略 / Dynamic Bus Lane (DBL) Activation Policy

● 目标 (The Goal):

根据实时数据库触发器转换车道为“公交专用”，以提高公交出行吞吐量。

Original: Improve transit throughput by converting lanes to "Bus-Only" based on real-time DB triggers.

● 详细任务 (Detailed Tasks):

- 车道模式：存储每条车道的状态（通用/公交专用）。

Original: Lane Schema: Store the state of every lane (General/Bus-Only).

- 延误监控：使用 TraCI 监控公交车延误；如果延误 > 5 分钟，在数据库中触发车道变更。

Original: Delay Monitor: Use TraCI to monitor bus delays; if delay > 5 mins, trigger a lane change in the DB.

- 基础设施更新：使用 TraCI 的 setLaneAllowed 物理更改 SUMO 中的车道访问权限。

Original: Infrastructure Update: Use TraCI's setLaneAllowed to physically change the lane access in SUMO.

- 公平性审计：使用 SQL 对比公交乘客与轿车司机的延误。

Original: Fairness Audit: Use SQL to compare the delay of bus passengers versus car drivers.

- 自动还原：当公交流量下降时，数据库触发器将车道还原为通用。

Original: Automatic Reversion: When bus volume drops, the DB trigger reverts the lane to general use.

● 性能评估规模与主题指标 (Evaluation Scale & Metrics):

- 路网：拥有 4 车道的 6km 干道走廊。

Original: Network: 6km arterial corridor with 4 lanes.

- 规模：25 辆公交车/小时；4,000 辆轿车/小时。

Original: Scale: 25 buses/hour; 4,000 passenger cars/hour.

- 时间：7,200秒（高峰窗口）。

Original: Time: 7,200s (Peak window).

- 主题指标：人员吞吐量 (人/小时) 和公共交通延误减少量

(%)。衡量“系统级效益”——公交车节省的时间是否超过轿车增加的延误？

Original: Topic Metrics: Person-Throughput (Pax/Hour) and Transit Delay Reduction (%). Measure the "Systemic Benefit"—did the bus time savings outweigh the car delay increase?

19. 重大中断下的递归图韧性 / Recursive Graph Resilience under Major Disruptions

- **目标 (The Goal):**

在路网发生多处失效时，使用递归图审计维持网络连通性。

Original: Maintain connectivity in a failing network using recursive graph audits.

- **详细任务 (Detailed Tasks):**

- 图邻接模式：将道路网格表示为一个图。

Original: Graph Adjacency Schema: Represent the road network as a graph.

- 故障注入：使用 TraCI 模拟多处道路关闭（例如洪水淹没）。

Original: Failure Injection: Use TraCI to simulate multiple road closures (e.g., flooding).

- 可达性分析：使用递归 CTE 寻找所有仍能从市中心到达的节点。

Original: Reachability Analysis: Use a Recursive CTE to find all nodes still reachable from the city center.

- 动态绕行：对于被孤立的车辆，查询数据库获取在网格完全碎裂前的“最后一条可行路径”。

Original: Dynamic Detouring: For isolated vehicles, query the DB for the "Last Possible Path" before the network fragments.

- 韧性得分：计算随时间推移保持连通的节点百分比。

Original: Robustness Score: Calculate the percentage of nodes connected over time.

- **性能评估规模与主题指标 (Evaluation Scale & Metrics):**

- 路网：城市级网络（2,500 个节点；6,000 条边）。

Original: Network: City-scale network (2,500 nodes; 6,000 edges).

- 规模：5,000 辆车。

Original: Scale: 5,000 vehicles.

- 时间：10,800秒灾难场景。

Original: Time: 10,800s disaster scenario.

- 主题指标：网络连通性指数和平均绕行长度增加量。分析“碎片化阈值”——在什么节点下，数据库查询无法再找到任何有效路径？

Original: Topic Metrics: Network Connectivity Index and Average Detour Length Increase. Analyze the "Threshold of Fragmentation"—at what point do DB queries fail to find any valid path?

20. 公共交通 “连通性保证” 同步 / Public Transit "Connectivity Guarantee" Sync

- **目标 (The Goal):**

确保乘客不会因为轻微的交通延误而错过 “末班车” 。

Original: Ensure passengers never miss the "last bus" due to minor traffic delays.

- **详细任务 (Detailed Tasks):**

- 时刻表模式：存储静态公交时刻表和实时支线接驳车位置。

Original: Schedule Schema: Store static schedules and real-time feeder shuttle locations.

- 同步查询：寻找支线接驳车上所有有 “换乘” 公交行程需求的乘客。

Original: Sync Query: Find all passengers on a "Feeder" vehicle who have a "Connecting" bus trip.

- 等候逻辑：如果支线接驳车晚点，数据库通过 TraCI 向换乘公交发出 “等候” 指令。

Original: Hold Logic: If the feeder is late, the DB sends a "Hold" command to the connecting bus via TraCI.

- 延误传播：使用 SQL 计算等候接驳车是否会导致该公交错过其下一个换乘连接。

Original: Delay Propagation: Use SQL to calculate if holding the bus will cause it to miss its next connection.

- 成功记录：记录 “获救的换乘连接” 数量。

Original: Success Logging: Track the number of "Saved Connections."

- **性能评估规模与主题指标 (Evaluation Scale & Metrics):**

- 路网：20 条公交线；10 条有轨电车线；150 个停靠站。

Original: Network: 20 bus lines; 10 tram lines; 150 stops.

- 规模：3,000 名换乘依赖型乘客。

Original: Scale: 3,000 transfer-dependent passengers.

- 时间：14,400秒（4小时模拟）。

Original: Time: 14,400s (4-hour simulation).

- 主题指标：换乘可靠性 (%) 和全系统乘客延误。评估换乘乘客 “总旅行时间” 的减少情况。

Original: Topic Metrics: Connection Reliability (%) and System-Wide Passenger Delay. Evaluate the reduction in "Total Travel Time" for transfer passengers.

21. 商业配送的实时“利润感知”路由 / Real-Time "Profit-Aware" Routing for Commercial Delivery

- **目标 (The Goal):**

综合考虑数据库中存储的燃料成本、通行费和时间惩罚，不仅针对时间进行路由优化，还针对配送路段的“净利润”进行优化。

Original: Optimize routing not just for time, but for the "Net Profit" of a delivery leg by factoring in fuel costs, toll fees, and time-penalties stored in a DB.

- **详细任务 (Detailed Tasks):**

- 成本模式：创建一个 Route_Economics 表，将路网的边 ID 映射到动态燃料消耗估算值和不同时间段的通行费。

Original: Cost Schema: Create a Route_Economics table mapping edge IDs to dynamic fuel consumption estimates and time-of-day tolling.

- 权重注入：实现一个基于 Python 的 Dijkstra 算法，该算法向数据库查询“利润权重”而不是标准的行程时间。

Original: Weight Injection: Implement a Python-based Dijkstra algorithm that queries the DB for "Profit-Weights" rather than standard travel times.

- 配送时间窗约束：使用 SQL 过滤掉违反特定订单“最迟送达时间”时间戳的候选路径。

Original: Delivery Window Constraint: Use SQL to filter potential routes that violate the "Must-Deliver-By" timestamp for a specific order.

- TraCI 重路由：监控实时交通拥堵；如果当前路线的“利润率”降至阈值以下，触发重新计算。

Original: TraCI Rerouting: Monitor real-time traffic jams; if the "Profit-Margin" of the current route drops below a threshold, trigger a re-calculation.

- 历史审计：记录每次完成行程的最终利润，以分析基于数据库成本预测的准确性。

Original: Historical Audit: Log the final profit of every completed trip to analyze the accuracy of the DB-based cost predictions.

- **性能评估规模与主题指标 (Evaluation Scale & Metrics):**

- 路网：具有 10 条收费主干道的 5 km * 5 km 城市网络。

Original: Network: 5 km * 5 km urban network with 10 tolled arterials.

- 规模：100 辆重型配送卡车；3,000 辆乘用车。

Original: Scale: 100 heavy delivery trucks; 3,000 passenger vehicles.

- 时间：7,200秒（高峰窗口）。

Original: Time: 7,200s (Peak window).

- 主题指标：每公里毛利润和按时送达率 (%)。

Original: Topic Metrics: Gross Profit per Kilometer and On-Time Delivery Rate (%).

22. 基于数据库交换的“基于压力”网络信号协调 / "Pressure-Based" Network Signal Coordination via DB-Exchange

• 目标 (The Goal):

实现去中心化的最大压力 (Max-Pressure) 控制，其中各个交叉口通过共享的本地数据库“交易/交换”排队状态信息。

Original: Implement a decentralized Max-Pressure control where intersections "Trade" queue-state information via a shared local database.

• 详细任务 (Detailed Tasks):

- 交叉口状态模式：在数据库中存储 10 个相邻交叉口的当前“流入量”和“流出量”计数。

Original: Intersection State Schema: Store the current "Inflow" and "Outflow" counts for 10 adjacent intersections in the DB.

- 压力计算：使用 SQL 视图计算每个信号相位的“压力权重”（流入量减去下游流出量）。

Original: Pressure Calculation: Use a SQL view to calculate the "Weight" of each signal phase (Inflow minus Downstream Outflow).

- 本地决策逻辑：为每个交叉口编写一个 Python 控制器，向数据库查询其自身和邻近交叉口的压力。

Original: Local Decision Logic: Write a Python controller for each intersection that queries its own and neighbors' pressure from the DB.

- TraCI 信号执行：将绿灯相位切换到相对压力最大的方向。

Original: TraCI Signal Actuation: Switch the green-phase to the direction with the highest relative pressure.

- 锁死审计：将“排队倒灌”（队列延伸到上游交叉口）的实例记录到 Network_Failure 表中。

Original: Gridlock Audit: Log instances of "Spillback" (queues reaching the upstream intersection) into a Network_Failure table.

• 性能评估规模与主题指标 (Evaluation Scale & Metrics):

- 路网：4 * 4 曼哈顿风格网格（16 个交叉口）。

Original: Network: 4 * 4 Manhattan-style grid (16 intersections).

- 规模：流经网格的 5,000 辆车辆。

Original: Scale: 5,000 vehicles flowing through the grid.

- 时间：7,200秒。

Original: Time: 7,200 s.

- 主题指标：全网平均速度和排队长度的标准差。

Original: Topic Metrics: Network-Wide Average Speed and Standard Deviation of Queue Lengths.

23. 拼车预约中的主动死锁预防 / Proactive Deadlock Prevention in Shared-Ride Reservations

● 目标 (The Goal):

当两辆车试图索要同一个高需求共享出行路段时，使用 SQL 事务隔离级别来防止“循环依赖”死锁。

Original: Use SQL transaction isolation levels to prevent "Circular Dependency" deadlocks when two vehicles attempt to claim the same high-demand shared-ride leg.

● 详细任务 (Detailed Tasks):

- 预约模式：设计一个 Ride_Legs 表，其中车辆 ID 为共享乘客“锁定”路径的特定路段。

Original: Reservation Schema: Design a Ride_Legs table where vehicle IDs "Lock" segments of a path for shared passengers.

- 死锁检测查询：编写一个 SQL 查询来识别新的预约是否会为现有乘客创建“无解/必败”的重路由。

Original: Deadlock Detection Query: Write a SQL query to identify if a new reservation will create a "No-Win" reroute for existing passengers.

- 事务回滚：使用 SQLite BEGIN IMMEDIATE 事务来确保在拼车匹配未能通过“时间窗”测试时，数据库状态能回滚。

Original: Transactional Rollback: Use SQLite BEGIN IMMEDIATE transactions to ensure that if a ride-share match fails the "Time-Window" test, the DB state reverts.

- TraCI 同步：只有在数据库事务成功提交后，才更新 SUMO 中的物理路径。

Original: TraCI Sync: Only update the physical route in SUMO once the DB transaction is successfully committed.

- 并发审计：记录高需求爆发期间的“事务重试次数”，以评估系统稳定性。

Original: Concurrency Audit: Log the number of "Transaction Retries" during high-demand bursts to evaluate system stability.

● 性能评估规模与主题指标 (Evaluation Scale & Metrics):

- 路网：2 km * 2 km 高密度中央商务区 (CBD)。

Original: Network: 2 km * 2 km high-density CBD.

- 规模：400 辆拼车出租车；2,000 个重叠的出租请求。

Original: Scale: 400 shared taxis; 2,000 overlapping trip requests.

- 时间：3,600秒（极高的需求爆发）。

Original: Time: 3,600 s (Massive demand spike).

- 主题指标：系统吞吐量（请求数/秒）和事务成功延迟。

Original: Topic Metrics: System Throughput (Requests/Sec) and Transaction Success Latency.

24. 以司机为中心的“返回家园”拼车 / "Return-to-Home" Driver-Centric Ride-Sharing

- **目标 (The Goal):**

通过使用数据库优先匹配在司机自家或目的地附近结束的共享出行请求，提高司机的留存率。

Original: Increase driver retention by using the DB to prioritize shared-ride requests that end near the driver's own home/destination.

- **详细任务 (Detailed Tasks):**

- 司机偏好模式：在数据库中存储每位司机的“家庭住址”和“期望收工时间”。

Original: Driver Preference Schema: Store every driver's "Home Location" and "Desired End-Time" in the DB.

- 方向性匹配：编写一个 SQL 查询，根据与司机回家路线的“角度偏差”来过滤乘客请求。

Original: Directional Matching: Write a SQL query that filters passenger requests based on the "Angular Deviation" from the driver's home path.

- 基于效用的派单：计算一个“匹配得分”，平衡乘客等待时间与司机回家距离的贴近度。

Original: Utility-Based Dispatching: Calculate a "Match-Score" that balances passenger wait-time with driver home-proximity.

- TraCI 路由：当司机接近其班次结束时，将最终的“回家”路径注入车辆中。

Original: TraCI Routing: Inject the final "Going-Home" route into the vehicle as it nears the end of its shift.

- 公平性审计：使用 SQL 计算车队中所有司机的“回家延误时间”。

Original: Fairness Audit: Use SQL to calculate the "Home-Arrival Delay" for all drivers in the fleet.

- **性能评估规模与主题指标 (Evaluation Scale & Metrics):**

- 路网：10km * 10 km 大都市区域。

Original: Network: 10km * 10 km metropolitan area.

- 规模：300 名司机（车辆）；1,200 个乘客请求。

Original: Scale: 300 drivers (cars); 1,200 passenger requests.

- 时间：10,800秒（晚班结束）。

Original: Time: 10,800 s (Evening shift end).

- 主题指标：平均回家时间误差（公里/分钟）与车队派单效率。

Original: Topic Metrics: Mean Home-Arrival Error (km/minutes) and Fleet Dispatch Efficiency.

25. 基于信号延时的网约拼车动态定价 / Dynamic Pricing for Ride-Sharing based on Signal Delay

● 目标 (The Goal):

使用数据库将拼车出行的价格与在其路线上预测的实时“信号灯延误”挂钩。

Original: Use the DB to link the cost of a ride-share trip to the real-time "Signal Delay" predicted on its route.

● 详细任务 (Detailed Tasks):

- 集成模式：将 Signal_States 和 Ride_Quotes 组合成一个统一的关系模型。

Original: Integrated Schema: Combine Signal_States and Ride_Quotes into a unified relational model.

- 延误预测：编写一个 SQL 查询，根据当前数据库的相位周期估算特定路线的“信号等候时间”。

Original: Delay Prediction: Write a SQL query that estimates the "Signal Wait" for a specific route based on current DB phase cycles.

- 激增价格逻辑：如果路线正在经历高强度的信号拥堵，在数据库中提高拼车价格。

Original: Surge-Logic: Increase the ride-share price in the DB if the route is experiencing high signal-induced congestion.

- TraCI 重路由：向乘客提供一条规避信号拥堵干道的“更便宜、耗时更长”的备选路线。

Original: TraCI Rerouting: Offer the passenger a "Cheaper, Longer" route that avoids the signal-clogged arterials.

- 收入与吞吐量审计：分析基于价格的重路由是否能改善交叉口的整体吞吐量。

Original: Revenue vs. Throughput Audit: Analyze if price-driven rerouting improves overall intersection throughput.

● 性能评估规模与主题指标 (Evaluation Scale & Metrics):

- 路网：具有 9 个交叉口的 3 km * 3 km 网格。

Original: Network: 3 km * 3 km grid with 9 intersections.

- 规模：500 辆拼车车辆；1,500 名对价格敏感的用户。

Original: Scale: 500 ride-share vehicles; 1,500 price-sensitive users.

- 时间：7,200秒。

Original: Time: 7,200 s.

- 主题指标：平均单程票价与交叉口饱和度 (V/C 比，流量/容量比)。

Original: Topic Metrics: Average Trip Fare and Intersection V/C (Volume-to-Capacity) Ratio.